

Titanic Survival Prediction Report

Charlotte Zhang | jz4027

Colab Link:

https://colab.research.google.com/drive/1_r-tYhJjCF90dVuJlJaibRkMj2zsU1xw#scrollTo=35951099

Github Repo Link: https://github.com/CharlotteZZh/AppliedML_hw1

Introduction pg 1

1.1 Problem Definition

1.2 Dataset Description

Data Cleaning pg 3-4

2.1 Initial Data Inspection

2.2 Cleaning Inconsistent Values

2.3 Train/Validation Split

Feature Engineering pg 4-7

3.1 Training Set Data Exploration After Splitting

3.2 Fare Transformation

3.3 Family Size

3.4 Passenger Title

3.5 Cabin and Deck Features

3.6 Ticket Prefix

3.7 Feature Selection

Model Training – Bernoulli pg 7-9

4.1 Preprocessing for Bernoulli

4.2 Training

4.3 Results

Model Training – Regressions pg 9-11

5.1 Preprocessing for Regression

5.2 Linear Regression

5.3 Ridge Regression

5.4 LASSO Regression

5.5 Results

Summary pg 11-12

6.1 Model Comparison

6.2 Effect of Smoothing

6.3 Final Model and Test Prediction

6.4 Conclusion and AI Disclosure

Introduction

1.1 Problem Definition

The goal of this project is to build an end-to-end machine learning workflow for predicting whether a passenger on the Titanic survived. The task is a binary classification problem, where the target variable Survived takes the value 1 if a passenger survived and 0 if the passenger did not survive. The project follows the complete machine learning pipeline: inspecting the raw data, cleaning missing and inconsistent values, engineering features, preprocessing numerical and categorical variables, training multiple models, evaluating their performance, and finally generating predictions for the unlabeled Titanic test dataset.

Two required modeling approaches were investigated:

1. Bernoulli Naive Bayes, a generative classification model.
2. Linear Regression, applied to the binary classification problem by interpreting its continuous output as a score and applying a threshold of 0.5.
3. In addition, Ridge and LASSO regression were tested to investigate regularization.

1.2 Dataset Description

The Titanic dataset contains information about passengers traveling on the Titanic. The training dataset contains 891 passengers and includes the target variable Survived. The test dataset contains 418 passengers without survival labels. The main variables include:

- PassengerId: unique passenger identifier.
- Survived: target variable indicating survival.
- Pclass: passenger class.
- Name: passenger name.
- Sex: passenger sex.
- Age: passenger age.
- SibSp: number of siblings or spouses aboard.
- Parch: number of parents or children aboard.
- Ticket: ticket number.
- Fare: ticket fare.
- Cabin: cabin information.
- Embarked: port of embarkation.

Data Cleaning

2.1 Initial Data Inspection

Before modifying the data, I inspected the structure, descriptive statistics, missing values, and duplicate rows. The training dataset contained 891 rows. The target distribution was approximately:

- 38.4% survived
- 61.6% did not survive

The Age variable had 177 missing observations (missing for approximately 19.2% of the training observations.), while Cabin had 687 missing observations (approximately 77.5% of the training observations, making it the most incomplete variable), and Embarked had only two missing values. The test set contained 86 missing Age values, 327 missing Cabin values, and one missing Fare value.

There were no duplicate rows in either the training or test dataset.

The missingness was not handled by simply dropping all incomplete rows. Dropping rows with missing Age or Cabin would unnecessarily remove a substantial portion of the dataset. Instead, missing values were handled using appropriate preprocessing methods later in the modeling pipeline. This approach preserves as many observations as possible while avoiding the introduction of information from the validation set into the training process.

TRAIN INFO

pandas 8 rows x 7 columns

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare	
	float64	float64	float64	float64	float64	float64	float64	
Scroll to top-left	891.00	891.00	891.00	714.00	891.00	891.00	891.00	
mean	446.00	0.3838	2.31	29.70	0.5230	0.3816	32.20	
std	257.35	0.4866	0.8361	14.53	1.10	0.8061	49.69	
min	1.00	0.00	1.00	0.4200	0.00	0.00	0.00	
25%	223.50	0.00	2.00	20.12	0.00	0.00	7.91	
50%	446.00	0.00	3.00	28.00	0.00	0.00	14.45	
75%	668.50	1.00	3.00	38.00	1.00	0.00	31.00	
max	891.00	1.00	3.00	80.00	8.00	6.00	512.33	

TEST INFO

pandas 8 rows x 6 columns

	PassengerId	Pclass	Age	SibSp	Parch	Fare	
	float64	float64	float64	float64	float64	float64	
count	418.00	418.00	332.00	418.00	418.00	417.00	
mean	1100.50	2.27	30.27	0.4474	0.3923	35.63	
std	120.81	0.8418	14.18	0.8968	0.9814	55.91	
min	892.00	1.00	0.1700	0.00	0.00	0.00	
25%	996.25	1.00	21.00	0.00	0.00	7.90	
50%	1100.50	3.00	27.00	0.00	0.00	14.45	
75%	1204.75	3.00	39.00	1.00	0.00	31.50	
max	1309.00	3.00	76.00	8.00	9.00	512.33	

Train

Int64

Test

Int64

Age

177

86

Cabin

687

327

Embark...

2

0

Fare

0

1

Name

0

0

Parch

0

0

Passen...

0

0

Pclass

0

0

Sex

0

0

SibSp

0

0

Survived

0

NA

Ticket

0

0

Duplica...

0

0

2.2 Cleaning Inconsistent Values

Before splitting the data, I applied cleaning operations that were based only on fixed rules rather than parameters learned from the data. Text fields including Name, Sex, Ticket, Cabin, and

Embarked were stripped of leading and trailing whitespace. The Sex field was converted to lowercase, while Embarked was converted to uppercase. I also checked numerical variables for impossible negative values. Negative values in Age, Fare, SibSp, or Parch were converted to missing values rather than being treated as valid observations. These operations are appropriate before the train/validation split because they do not calculate statistics from the dataset. They simply enforce basic consistency rules. Before and after cleaning examples where trailing whitespace had been removed:

Dataset string	PassengerId int64	Column string	Before string	After string
Train	16	Name	'Hewlett, Mrs. (Mary D Kingcome) '	'Hewlett, Mrs. (Mary D Kingcome)'
Train	858	Name	'Daly, Mr. Peter Denis '	'Daly, Mr. Peter Denis'
Test	1167	Name	'Bryhl, Miss. Dagmar Jenny Ingeborg '	'Bryhl, Miss. Dagmar Jenny Ingeborg'
Test	1222	Name	'Davies, Mrs. John Morgan (Elizabeth Agnes Mary White) '	'Davies, Mrs. John Morgan (Elizabeth Agnes Mary White)'

2.3 Train/Validation Split

After the initial fixed-rule cleaning, the labeled training data were divided into training and validation sets, as the test dataset had no ground-truth labels and therefore could not be used for model evaluation. An 80/20 split was used with a fixed random state of 42. Stratification by the target variable was also applied to maintain an approximately consistent survival distribution across the training and validation subsets. The resulting datasets were: Training set has 712 observations; Validation set has 179 observations

The validation set was kept separate from model fitting and preprocessing. This allows the models to be evaluated on observations that were not used during training. This separation is especially important for imputation, scaling, and categorical encoding because these operations can otherwise introduce information leakage.

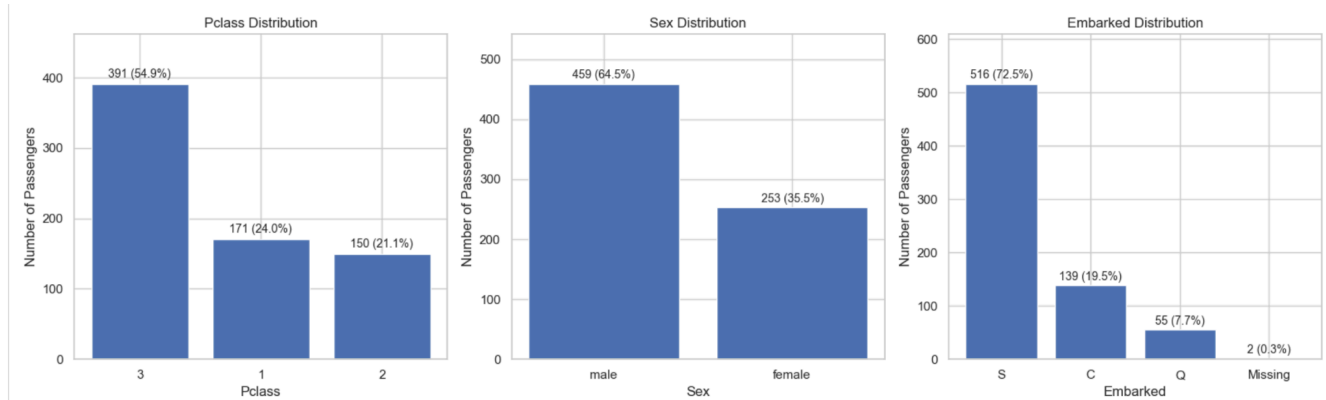
Feature Engineering

3.1 Training Set Data Exploration After Splitting

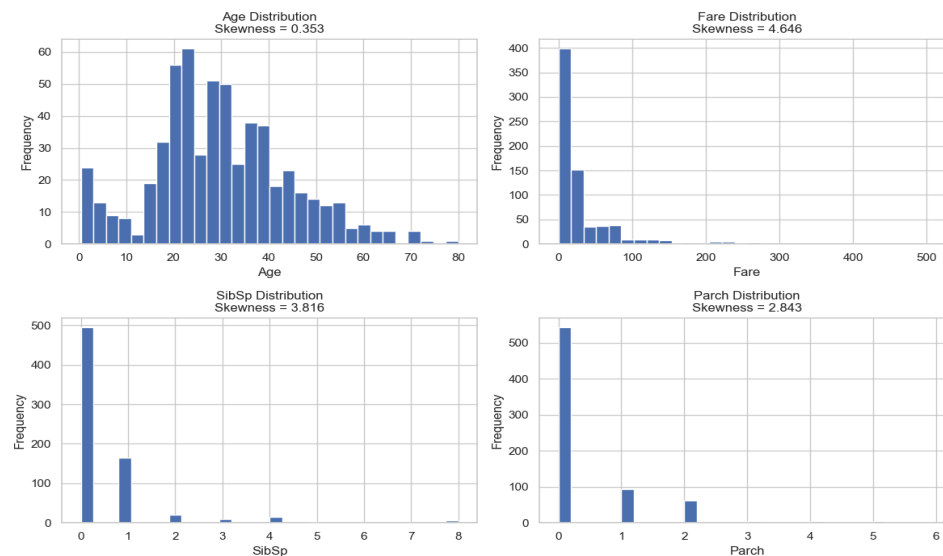
Missing Values: The largest amounts of missing data were found in Cabin and Age. Cabin had 552 missing values (77.5%), while Age had 137 (19.2%). Only 2 Embarked values were missing.

	Missing Count int64	Missing Percentage float64
Cabin	552	77.53
Age	137	19.24
Embark...	2	0.2809
Passen...	0	0.00
Pclass	0	0.00
Name	0	0.00
Sex	0	0.00
SibSp	0	0.00
Parch	0	0.00
Ticket	0	0.00
Fare	0	0.00

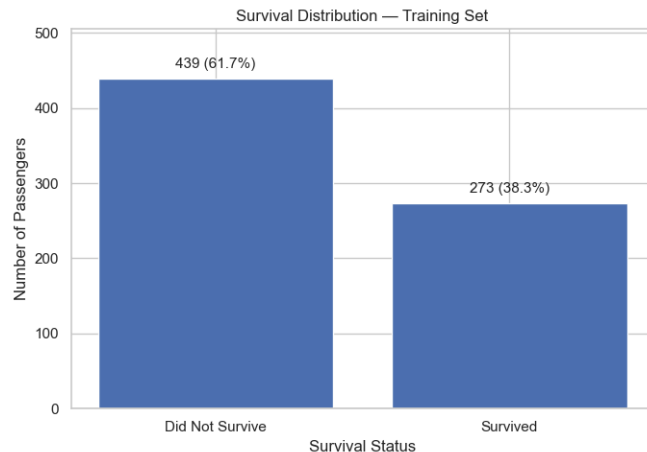
Categorical Variables: The main categorical variables, including Pclass, Sex, and Embarked, were examined to understand their distributions. The training set contains passengers from all three classes, more males than females, and passengers primarily embarking from Southampton.



Numerical Variables: The distributions of Age, Fare, SibSp, and Parch were examined to identify skewness and potential transformations. Age was relatively symmetric, with a skewness of 0.353. In contrast, Fare, SibSp, and Parch were strongly right-skewed, with skewness values of 4.646, 3.816, and 2.843, respectively.



Target Variable: The Survived variable was also examined. More passengers did not survive than survived, indicating a moderate class imbalance. Therefore, model evaluation later considers not only accuracy but also precision, recall, F1-score, and the confusion matrix.



Overall, this exploration identified the key characteristics of the training data and guided the preprocessing and feature-engineering decisions described in the next section.

3.2 Fare Transformation

Because Fare was strongly right-skewed (skewness = 4.646), a logarithmic transformation was applied: $\text{FareLog} = \log(1 + \text{Fare})$. This reduces the influence of extremely high fares while preserving the ordering of the observations.

3.3 Family Size

A FamilySize feature was created by combining SibSp and Parch: $\text{FamilySize} = \text{SibSp} + \text{Parch} + 1$. The additional 1 represents the passenger. This provides a single measure of the size of the passenger's immediate family group aboard the ship. An IsAlone indicator was created from FamilySize. Passengers with a FamilySize of 1 were classified as traveling alone, while passengers with larger family sizes were classified as traveling with family.

3.4 Passenger Title

The passenger's Name was used to extract their title, such as Mr, Miss, Mrs, and Master. Rare titles were grouped into a Rare category, while equivalent titles such as Mlle, Ms, and Mme were standardized to Miss or Mrs. This converts the largely unique Name field into a smaller categorical feature that can capture useful demographic information.

3.5 Cabin and Deck Features

The Cabin variable contained a large amount of missing information. Instead of using the original cabin number directly, the first character of the cabin value was extracted as Deck.

Missing cabin information was represented as Unknown. A CabinKnown indicator was also created to distinguish passengers with recorded cabin information from those without it.

3.6 Ticket Prefix

The Ticket variable was transformed into a TicketPrefix feature by extracting the non-numeric portion of the ticket. Tickets without a meaningful prefix were assigned the category NUMERIC.

3.7 Feature Selection

After feature engineering, the original PassengerId, Name, Ticket, Cabin, SibSp, Parch, and Fare variables were removed. The final features consisted of Age, FareLog, and FamilySize as numerical variables, and Pclass, Sex, Embarked, Title, Deck, TicketPrefix, and IsAlone as categorical variables. Some of these variables were removed because their information had already been captured by engineered features. For example, SibSp and Parch were combined into FamilySize, while Fare was replaced by FareLog. Removing the original variables also helps reduce multicollinearity, since keeping both the original variables and their derived versions could introduce redundant information and highly correlated predictors.

Model Training – Bernoulli

4.1 Preprocessing for Bernoulli

The categorical variables were one-hot encoded, while the numerical variables were first imputed using the training-set median, then discretized into five quantile-based bins and one-hot encoded. Median imputation was used for numerical variables because it is robust to outliers and skewed distributions, while mode imputation was used for categorical variables because they cannot be meaningfully averaged and the most frequent category provides a simple replacement.

- Numerical variables: median imputation → quantile binning → one-hot encoding
- Categorical variables: mode imputation → one-hot encoding

This converted the numerical variables into binary indicators appropriate for Bernoulli Naive Bayes. Missing categorical values were imputed using the training-set mode. All preprocessing parameters were fitted only on the training data and then applied to the validation data to prevent data leakage. The resulting Bernoulli Naive Bayes feature matrix contained 60 binary features. Below shows the before and after:

BernoulliNB numerical imputation examples:

nb_imputation_examples 5 rows x 3 columns			
PassengerId int64	Age Before float64	Age After Impu... float64	
692	693	NaN	28.50
481	482	NaN	28.50
527	528	NaN	28.50
557	558	NaN	28.50
828	829	NaN	28.50

BernoulliNB numerical features after quantile binning:

numeric_binned_df 5 rows x 12 columns				
Age_0.0 float64	Age_1.0 float64	Age_2.0 float64	Age_3.0 float64	
0.00	0.00	1.00	0.00	
0.00	0.00	1.00	0.00	
0.00	0.00	1.00	0.00	
1.00	0.00	0.00	0.00	
0.00	0.00	0.00	1.00	

BernoulliNB categorical encoding examples:

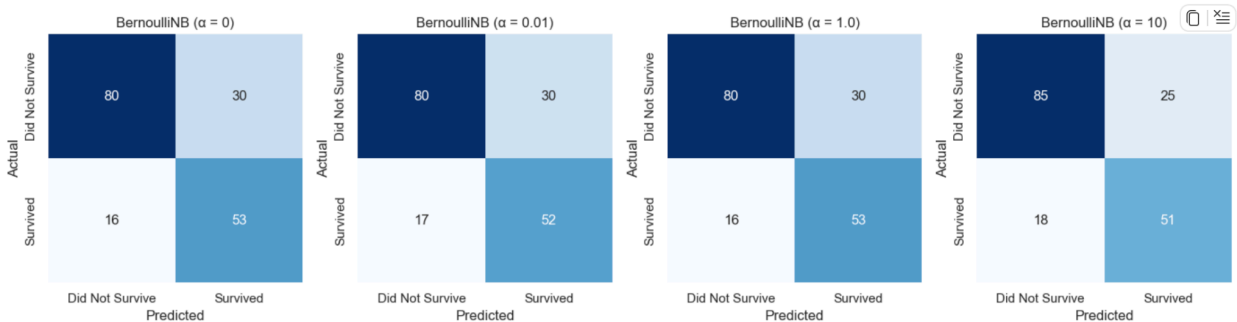
nb_encoding_examples 2 rows x 4 columns			
PassengerId int64	Sex Before string	Sex_female float64	Sex_male float64
692	male	0.00	1.00
855	female	1.00	0.00

4.2 Training

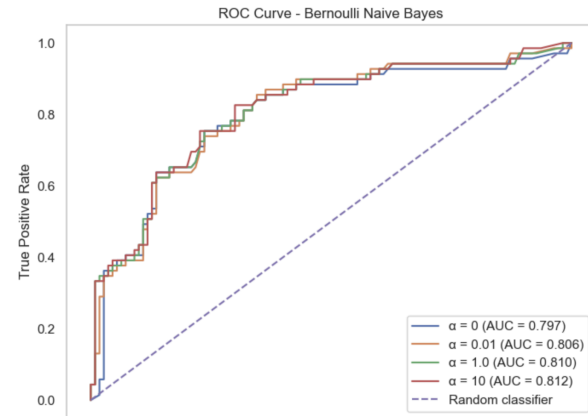
Bernoulli Naive Bayes was trained using four values of the Laplace smoothing parameter $\alpha = 0, 0.01, 1.0$. Using multiple values allowed the effect of smoothing on model performance to be evaluated.

4.3 Results

The Bernoulli Naive Bayes results show that changing the smoothing parameter α affected model performance. The best overall performance was achieved with $\alpha = 10$, which produced 75.98% accuracy, 67.11% precision, 70.34% recall, 70.34% F1-score, and an AUC of 0.8121. Its confusion matrix shows 85 true negatives and 51 true positives, with 25 false positives and 18 false negatives. The ROC curves show that all α values performed better than random classification, while $\alpha = 10$ achieved the highest AUC.



	Alpha float64	Accuracy float64	Precision float64	Recall float64	F1 float64	AUC float64
0	0.00	0.7430	0.6386	0.7681	0.6974	0.7969
1	0.0100	0.7374	0.6341	0.7536	0.6887	0.8057
2	1.00	0.7430	0.6386	0.7681	0.6974	0.8098
3	10.00	0.7598	0.6711	0.7391	0.7034	0.8121



Model Training – Regressions

5.1 Preprocessing for Regression

The regression models used a different numerical preprocessing approach from Bernoulli Naive Bayes. Both used the same basic missing-value treatment: numerical variables were imputed using the training-set median, while categorical variables were imputed using the training-set mode and then one-hot encoded. The main difference was the treatment of numerical variables. For Bernoulli Naive Bayes, numerical variables were converted into binary features through quantile binning. For the regression models, the numerical variables remained continuous and were standardized using StandardScaler. This preserves the numerical relationships between observations and makes the features comparable in scale. The regression preprocessing therefore consisted of:

- Numerical variables: median imputation → standardization
- Categorical variables: mode imputation → one-hot encoding

All preprocessing parameters were fitted only on the training data and then applied to the validation data to prevent data leakage. The resulting regression feature matrix contained 44 features.

Age: 137 missing before; 0 missing after
Embarked: 2 missing before; 0 missing after

pandas 7 rows x 5 columns						
	PassengerId int64	Feature string	Before float64	After mixed	Rule string	
0	693	Age		NaN 28.50	Training median	
1	482	Age		NaN 28.50	Training median	
2	528	Age		NaN 28.50	Training median	
3	558	Age		NaN 28.50	Training median	
4	829	Age		NaN 28.50	Training median	
5	830	Embarked		NaN S	Training mode	
6	62	Embarked		NaN S	Training mode	

Numerical scaling: five training passengers

scaling_examples 5 rows x 7 columns								Open in Data Explorer					
	PassengerId int64	:	Age before sca... float64	:	Age after sca... float64	:	FareLog before... float64	:	FareLog after ... float64	:	FamilySize bef... float64	:	FamilySize aft... float64
692	693	:	28.50	:	-0.0811	:	4.05	:	1.12	:	1.00	:	-0.55
481	482	:	28.50	:	-0.0811	:	0.00	:	-3.01	:	1.00	:	-0.55
527	528	:	28.50	:	-0.0811	:	5.41	:	2.51	:	1.00	:	-0.55
855	856	:	18.00	:	-0.8878	:	2.34	:	-0.6270	:	2.00	:	0.07
801	802	:	31.00	:	0.1109	:	3.31	:	0.3619	:	3.00	:	0.70

Categorical encoding: one passenger per observed Sex category

encoding_examples 2 rows x 3 columns					 Open in Data Explorer
	PassengerId int64	:	Sex before enc... string	:	Sex_male float64
692	693	:	male	:	1.00
855	856	:	female	:	0.00

5.2 Linear Regression

Linear Regression was applied to the binary Survived target as required by the assignment. Although Linear Regression normally predicts a continuous value, its output was interpreted as a prediction score for this classification problem. A threshold of 0.5 was used to convert the continuous prediction into a binary class:

- Prediction $\geq 0.5 \rightarrow$ Survived
- Prediction $< 0.5 \rightarrow$ Did not survive

Linear Regression achieved an accuracy of 83.80%, precision of 80.30%, recall of 76.81%, F1-score of 78.52%, and AUC of 0.8706.

5.3 Ridge Regression

Ridge Regression was also evaluated to investigate the effect of L2 regularization. An α value of 1.0 was used. Ridge achieved an accuracy of 82.12%, precision of 79.37%, recall of 72.46%, F1-score of 75.76%, and AUC of 0.8693.

5.4 LASSO Regression

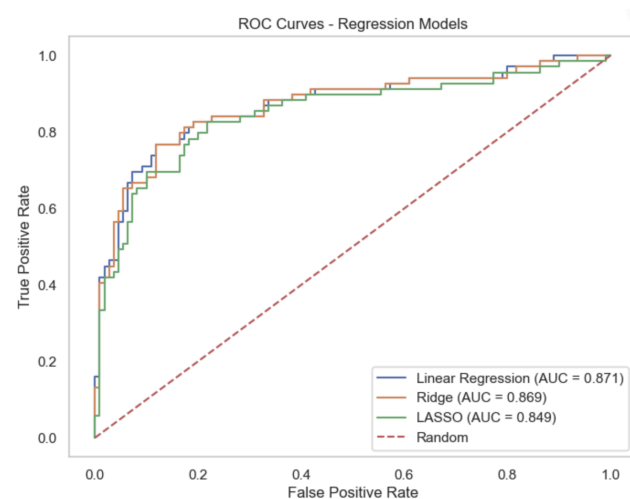
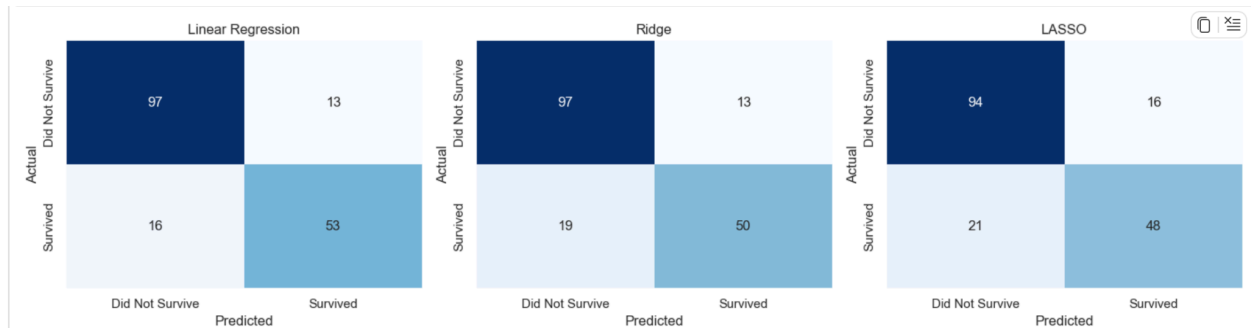
LASSO Regression was evaluated using L1 regularization with $\alpha = 0.01$. LASSO achieved an accuracy of 79.33%, precision of 75.00%, recall of 69.57%, F1-score of 72.18%, and AUC of 0.8487.

5.5 Results

Linear Regression achieving the best results across all evaluation metrics. It obtained 83.80% accuracy, 80.30% precision, 76.81% recall, 78.52% F1-score, and an AUC of 0.8706. Ridge performed slightly worse, while LASSO had the lowest performance among the three. The confusion matrices show that Linear Regression correctly classified 97 passengers who did not survive and 53 who survived, with 13 false positives and 16 false negatives. The ROC curves also

show that all three models performed substantially better than random classification, with Linear Regression having the highest AUC.

regression_results_df 3 rows x 6 columns						Open in Jupyter	
	Model string	Accuracy float64	Precision float64	Recall float64	F1 float64	AUC float64	
0	Linear Regression	0.8380	0.8030	0.7681	0.7852	0.8706	
1	Ridge	0.8212	0.7937	0.7246	0.7576	0.8693	
2	LASSO	0.7933	0.7500	0.6957	0.7218	0.8487	



Summary

6.1 Model Comparison

Linear Regression performed best, achieving the highest accuracy (83.80%), F1-score (78.52%), and AUC (0.8706). Ridge and LASSO performed slightly worse, with LASSO having the lowest overall performance. Linear Regression likely performed better because it retained the continuous numerical features rather than converting them into binary bins.

6.2 Effect of Smoothing

For Bernoulli Naive Bayes, $\alpha = 10$ performed best, while $\alpha = 0$ (no smoothing) performed worse. Smoothing helps prevent zero or very small probability estimates, which can make predictions more stable. However, too much smoothing can also reduce useful differences between features, so the best value depends on the dataset.

6.3 Final Model and Test Prediction

Linear Regression was selected as the final model because it had the strongest validation performance across all metrics. It was retrained on all 891 training observations and used to predict the 418 test observations, using a 0.5 threshold. The first 10 predictions were:

	PassengerId int64	:	Survived int64	:
0		892		0
1		893		1
2		894		0
3		895		0
4		896		1
5		897		0
6		898		1
7		899		0
8		900		1
9		901		0

6.4 Conclusion and AI Disclosure

Overall, the results show that feature engineering, appropriate preprocessing, and model selection significantly affected performance. Linear Regression provided the strongest results on the validation set and was therefore chosen for the final Titanic test predictions.

I used ChatGPT and Codex as supporting tools throughout this project. The overall logic, workflow, ordering of steps, feature engineering decisions, model choices, evaluation approach, and decisions about what to focus on were my own. I used AI primarily for debugging, explaining concepts, and generating plotting and parts of model-training code based on my instructions. I reviewed, ran, and modified the generated code and made the final decisions and interpretations myself. AI tools did not independently complete the project or determine its methodology.